

Transformers, Modern MLP Architectures and Foundation Models

Prof. Artur Jordão

Introduction

Transformers

- The role of context

*The **chicken** didn't cross the road because **it** was too tired*



*The chicken didn't cross the **road** because **it** was too wide*



- Without reading the full sentence, can you tell what *it* refers to?
- We need a representation that encodes information about both *chicken* and *road*, enabling the model to use context to infer what *it* refers to

Introduction

Transformers

- Transformers-like models stand as the modern paradigm for solving cognitive tasks (Bengio et al., 2025)
- Surprisingly, these models perform on par or better than convolutional networks (Dosovitskiy et al., 2021)
 - Convolutional models have dominated deep learning for computer vision tasks

Model	CIFAR-10	ImageNet
ResNet	93.57	80.62
NASNet	97.60	74.00
Transformer	99.50	88.55

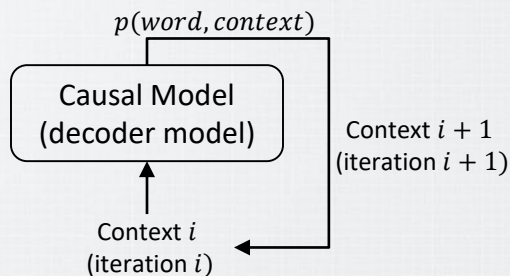
Bengio et al. *International AI Safety Report*. 2025

Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. International Conference on Learning Representations (ICLR), 2021

Preliminaries

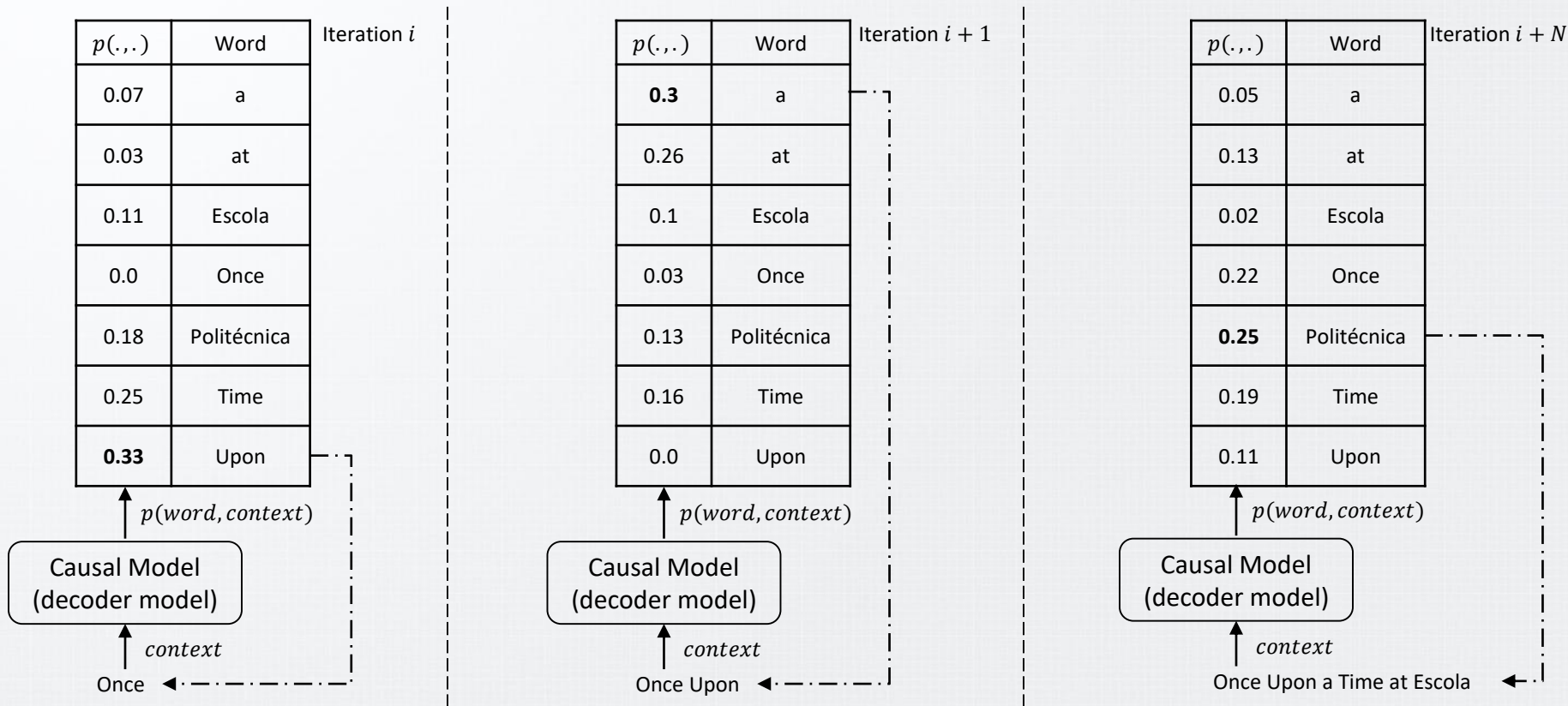
Transformers

- Autoregressive models given a sequence of input tokens and predict output tokens **one by one** by conditioning on the prior context
 - Also known as left-to-right language models or **causal** models



Preliminaries - Autoregressive Process

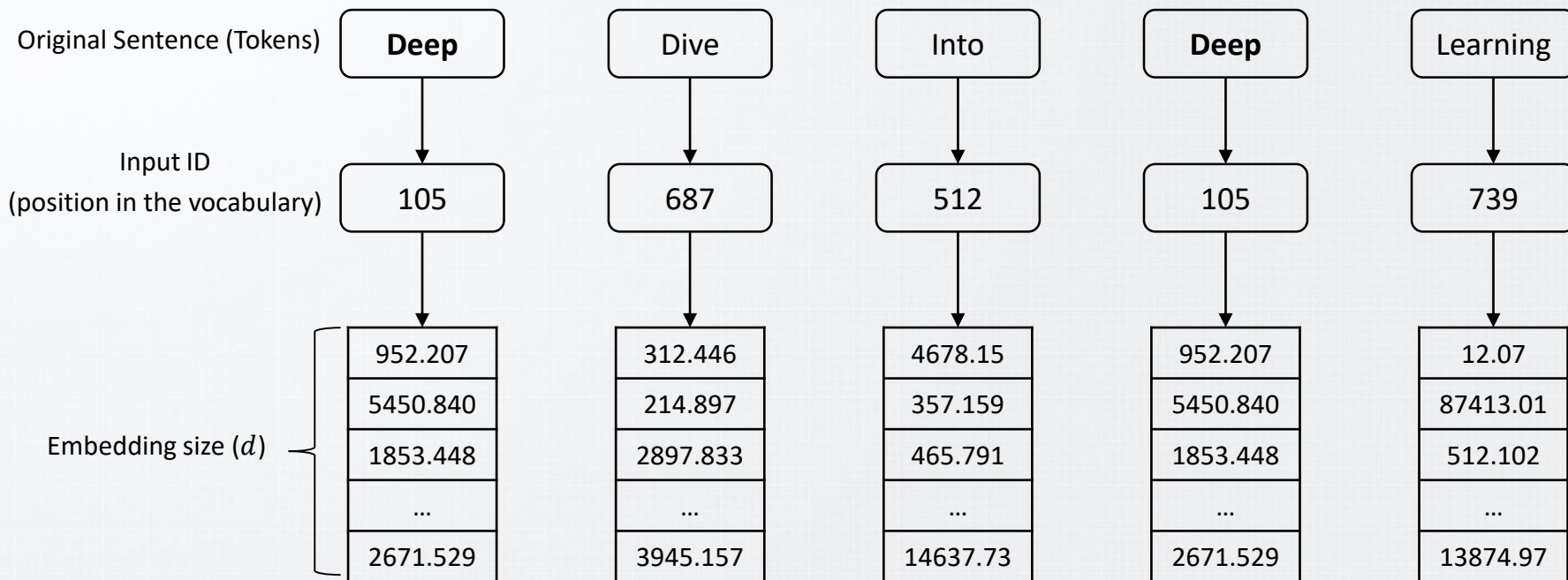
Transformers



Preliminaries - Embedding

Transformers

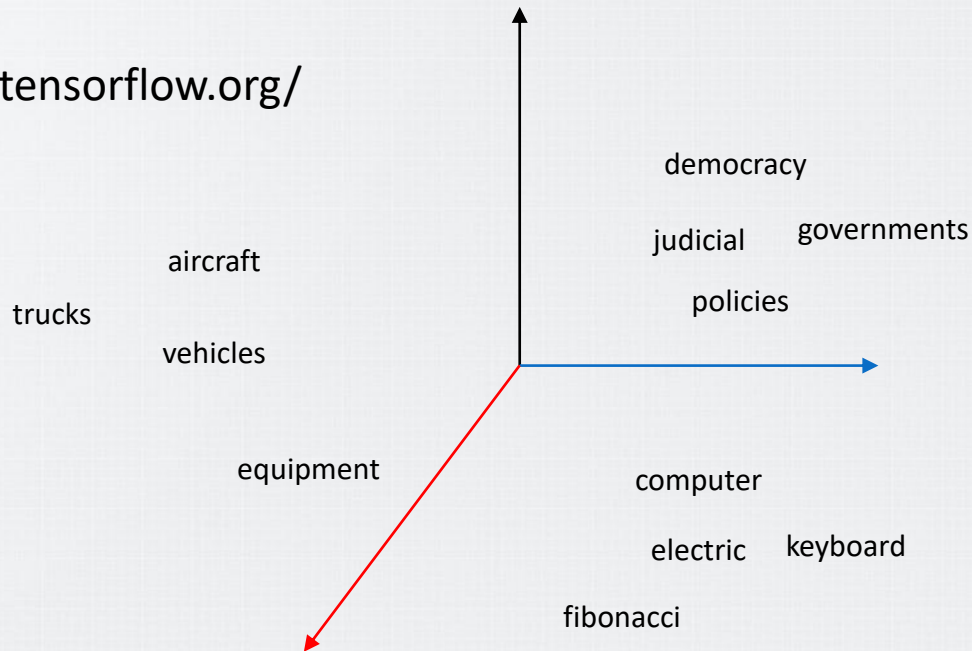
- Examples at <https://platform.openai.com/tokenizer>



Preliminaries - The Role of Embedding

Transformers

- The embedding maps words into a continuous vector space
 - Semantically **similar** words are positioned **closer** together
 - It helps models capture meaning and relationships between words effectively
- Examples at <https://projector.tensorflow.org/>



Transformer Architecture

Self-Attention (SA)

Transformer Architecture

- At the heart of Transform lies the self-attention mechanism
 - Ability to learn **pairwise relationships** between **all** tokens/features in a given sequence
 - Attention to itself
- A self-attention, $SA(\cdot)$, block takes n inputs $x_1, x_2, \dots, x_n$ and returns n output vectors of the same size
 - Each x_i has dimensions $d \times 1$
- <https://www.transformer101.com/self-attention>
 - Self-Attention Visualizer

Query, Key and Value

Transformer Architecture

- There are three different roles that each input embedding plays during the course of the attention process
- Query
 - The current element to compare against the preceding inputs
- Key
 - A preceding input for comparison with the current element, to determine a similarity weight
- Value
 - The preceding element that receives a weight and contributes to the output computation for the current element

Queries and Keys

Transformer Architecture

- Queries (q) and keys (k) are linear transformations
 - $q_i = W^q x_i + b^q$
 - $k_i = W^k x_i + b^k$
- The queries and keys must have the same dimensions

Values

Transformer Architecture

- The model computes a set of values for each input in terms of $v_i = W^v x_i + b^v$
 - W^v indicates (learnable) weights
 - b^v indicates the bias
- Note that **all inputs** $x_i \in \mathbb{R}^d$ share the weights $W^v \in \mathbb{R}^{d \times d}$ and biases $b^v \in \mathbb{R}^d$

Attention

Transformer Architecture

- The scalar weight $a[x_i, x_n]$ is **the attention that the n th output pays to input x_i**
- The n weights $a[, x_n]$ are non-negative and sum to one
- The attention weights $a[x_i, x_n]$ combine the values from different inputs
- $a[x_i, x_n] = \text{Softmax}(q_n k_n^T)$

Matrix Form

Transformer Architecture

- We can put Values, Keys and Queries into a compact form
 - Consider that x_i corresponds to one row of the $n \times d$ matrix X

- $V = XW^v + b^v \mathbf{1}^T$

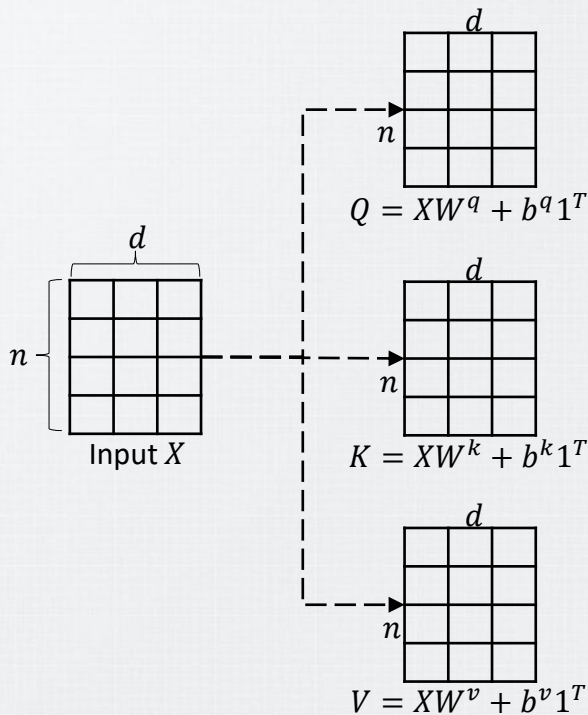
- Values

- $Q = XW^q + b^q \mathbf{1}^T$

- Queries

- $K = XW^k + b^k \mathbf{1}^T$

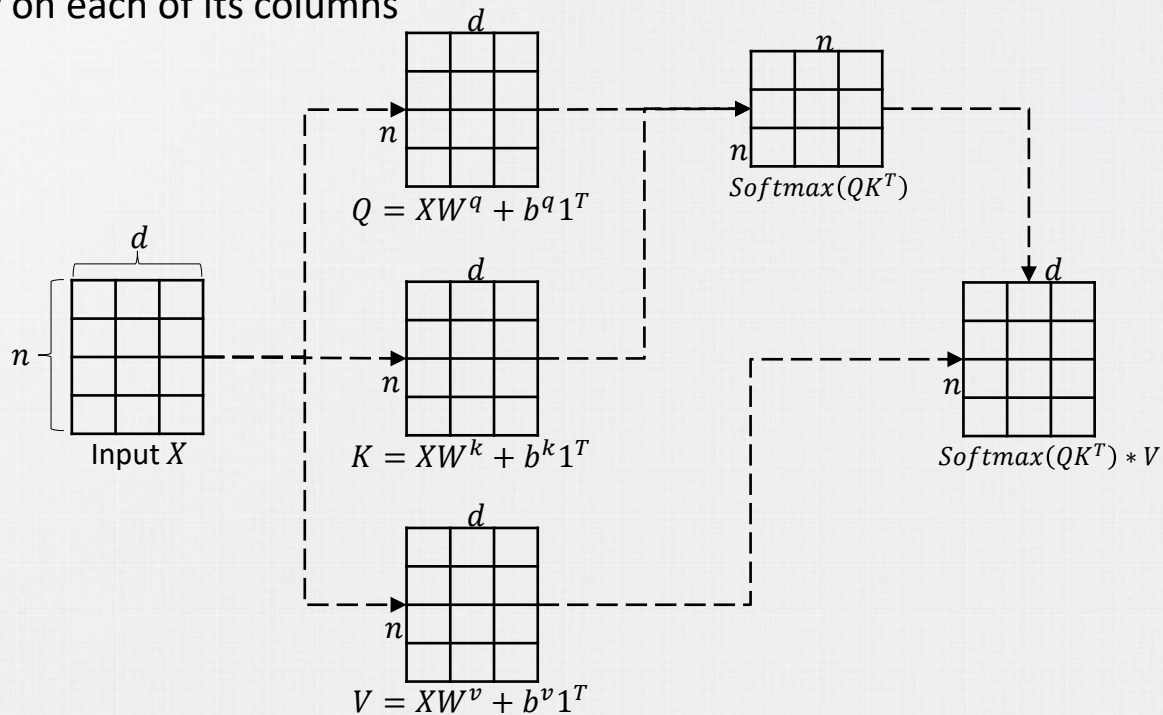
- Keys



Self-Attention (SA)

Transformer Architecture

- $SA = \text{Softmax}(QK^T) * V$
 - In this matrix form, the $\text{Softmax}(\cdot)$ function takes a matrix and performs the softmax operation independently on each of its columns



Masked Attention

Transformer Architecture

- Masking out the future
 - Guessing the next word is easy if you already know it

This eliminates any knowledge of words that follow in the sequence (the softmax will turn to zero)

$$\begin{array}{c} \text{Mask} \end{array} \left[\begin{array}{c} Q \\ \begin{array}{|c|} \hline q1 \\ \hline q2 \\ \hline q3 \\ \hline q4 \\ \hline \end{array} \end{array} \right] \times \begin{array}{c} K^T \\ \begin{array}{|c|c|c|c|} \hline k1 & k2 & k3 & k4 \\ \hline \end{array} \end{array} = \begin{array}{c} QK^T \\ \begin{array}{|c|c|c|c|} \hline q1 \cdot k1 & q1 \cdot k2 & q1 \cdot k3 & q1 \cdot k4 \\ \hline q2 \cdot k1 & q2 \cdot k2 & q2 \cdot k3 & q2 \cdot k4 \\ \hline q3 \cdot k1 & q3 \cdot k2 & q3 \cdot k3 & q3 \cdot k4 \\ \hline q4 \cdot k1 & q4 \cdot k2 & q4 \cdot k3 & q4 \cdot k4 \\ \hline \end{array} \end{array} = \begin{array}{c} QK^T \text{ masked} \\ \begin{array}{|c|c|c|c|} \hline q1 \cdot k1 & -\infty & -\infty & -\infty \\ \hline q2 \cdot k1 & q2 \cdot k2 & -\infty & -\infty \\ \hline q3 \cdot k1 & q3 \cdot k2 & q3 \cdot k3 & -\infty \\ \hline q4 \cdot k1 & q4 \cdot k2 & q4 \cdot k3 & q4 \cdot k4 \\ \hline \end{array} \end{array} \times \begin{array}{c} V \\ \begin{array}{|c|} \hline v1 \\ \hline v2 \\ \hline v3 \\ \hline v4 \\ \hline \end{array} \end{array} = \begin{array}{c} A \\ \begin{array}{|c|} \hline a1 \\ \hline a2 \\ \hline a3 \\ \hline a4 \\ \hline \end{array} \end{array}$$

Scaled Dot Product Self-Attention

Transformer Architecture

- The dot products in the attention computation can have large magnitudes
 - It moves the arguments to the softmax function into a region where the largest value completely dominates
 - Small changes to the inputs to the softmax function now have little effect on the output (i.e., the gradients are very small)
- We can scale the dot products by the square root of the dimension d of the queries and keys
 - $SA = \text{Softmax} \left(\text{mask} \left(\frac{QK^T}{\sqrt{d}} \right) \right) * V$
 - This is known as **scaled dot product self-attention**

Multi-Head Attention

Transformer Architecture

- To enable the model to learn different representations, the architecture applies the self-attention mechanism multiple (h) times for the same input X
 - We name the h th self-attention as head
- Each head has its own set of weights (and biases)
 - W^{vh}, W^{qv}, W^{kv}
- The self-attention to the head h is: $Softmax\left(mask\left(\frac{Q_h K_h^T}{\sqrt{d}}\right)\right) * V_h$
- Finally, we concatenate and project onto a learnable matrix W^o
 - $Multihead(Q, K, V) = Concat(head_1, head_2, \dots, head_h)W^o$

Complexity

Transformer Architecture

- Advantages of attention layers over recurrent and convolutional networks
 - Lower computational complexity
 - Higher connectivity: especially useful for learning long-term dependencies in sequences
- Definitions
 - n , sequence length
 - k , kernel size (convolutional layers)
 - d , representation dimension

Layer Type	Complexity per Layer
Recurrent	$O(n \times d^2)$
Convolutional	$O(k \times n \times d^2)$
Self-Attention	$O(n^2 \times d)$

Positional Encoding

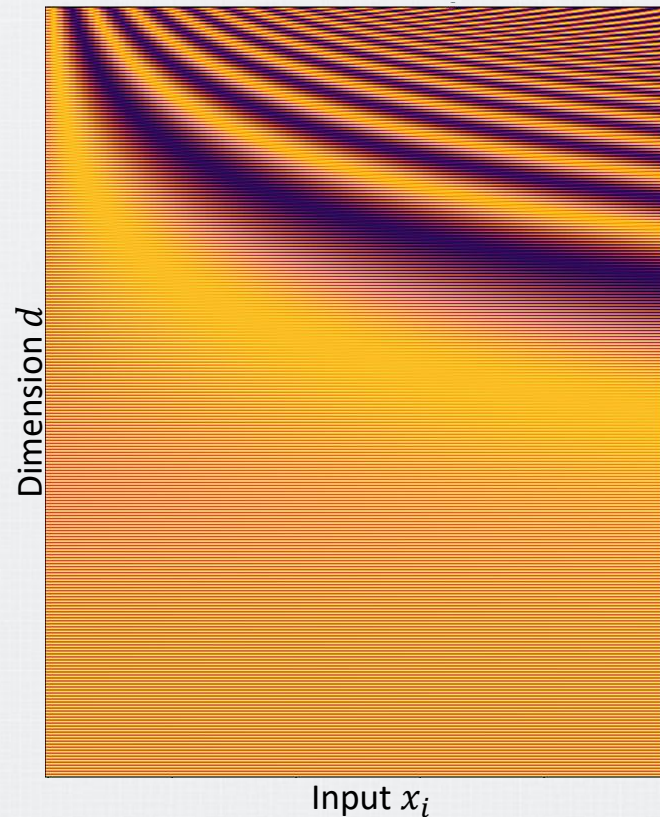
Transformer Architecture

- The previous components we have seen so far are unable to capture **relative positions**: they contain no recurrence or convolution
 - The computation is the same regardless of the order of the inputs x_n
 - It is equivariant with respect to input permutations
- The positional encoding injects some information about the **relative or absolute position** of the input in the sequence
 - This component plays a role in Transformers-like models

Positional Encoding

Transformer Architecture

- Sinusoidal pattern (sine and cosine functions)
 - $PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$
 - $PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$
 - pos is the index in the sequence and i is the i th position in d ($i \in \{0, 1, 2 \dots (|d| - 1)\}$)



Positional Encoding

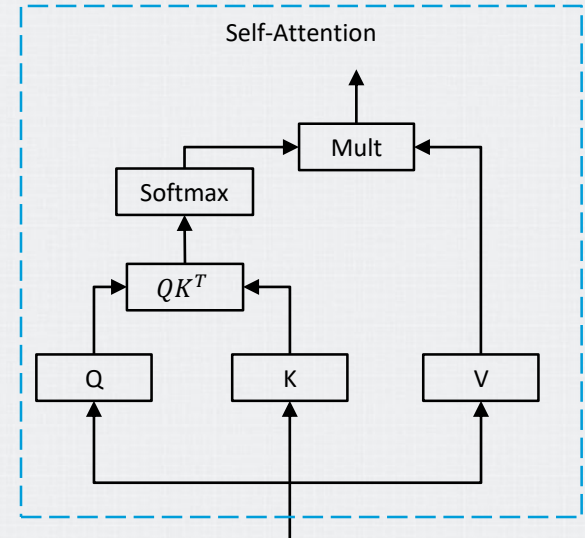
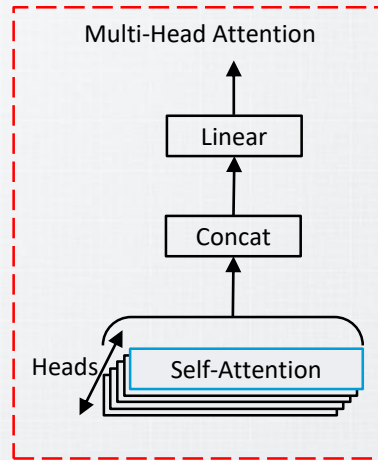
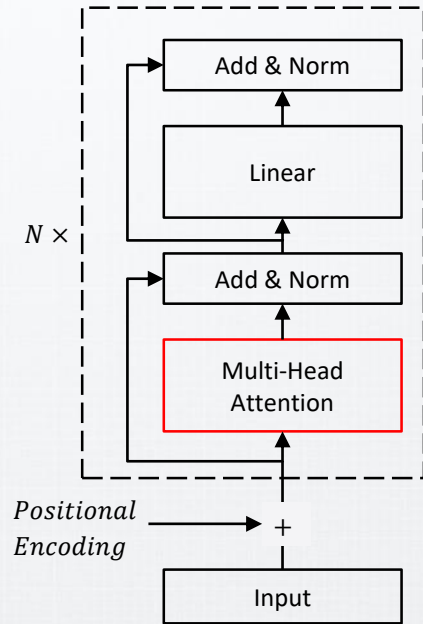
Transformer Architecture

- Sinusoidal pattern (sine and cosine functions)
 - $PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$
 - $PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$
 - pos is the index in the sequence and i is the i th position in d ($i \in \{0, 1, 2 \dots (|d| - 1)\}$)

Sequence	pos	$i = 0$	$i = 1$	$i = 2$	$i = 3$
I	0	$PE_{(00)} = \sin(0) = 0$	$PE_{01} = \cos(0) = 1$	$PE_{(02)} = \sin(0) = 0$	$PE_{(03)} = \cos(0) = 1$
Am	1	$PE_{10} = \sin(1) = 0.84$	$PE_{11} = \cos(1) = 0.54$	$PE_{(12)} = \sin(0.01) = 0.009$	$PE_{(13)} = \cos(0.01) = 0.9999$
A	2	$PE_{20} = \sin(2) = 0.909$	$PE_{21} = \cos(2) = -0.416$	$PE_{(22)} = \sin(0.02) = 0.019$	$PE_{(23)} = \cos(0.02) = 0.9998$
Robot	3	$PE_{30} = \sin(3) = 0.14$	$PE_{31} = \cos(3) = -0.989$	$PE_{32} = \sin(0.03) = 0.0299$	$PE_{33} = \cos(0.03) = 0.9995$

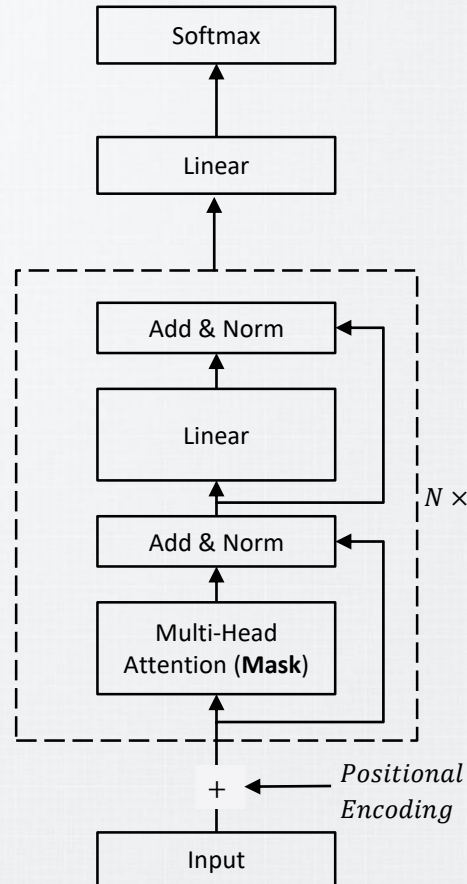
Overall Architecture

Transformer Architecture



Overall Architecture

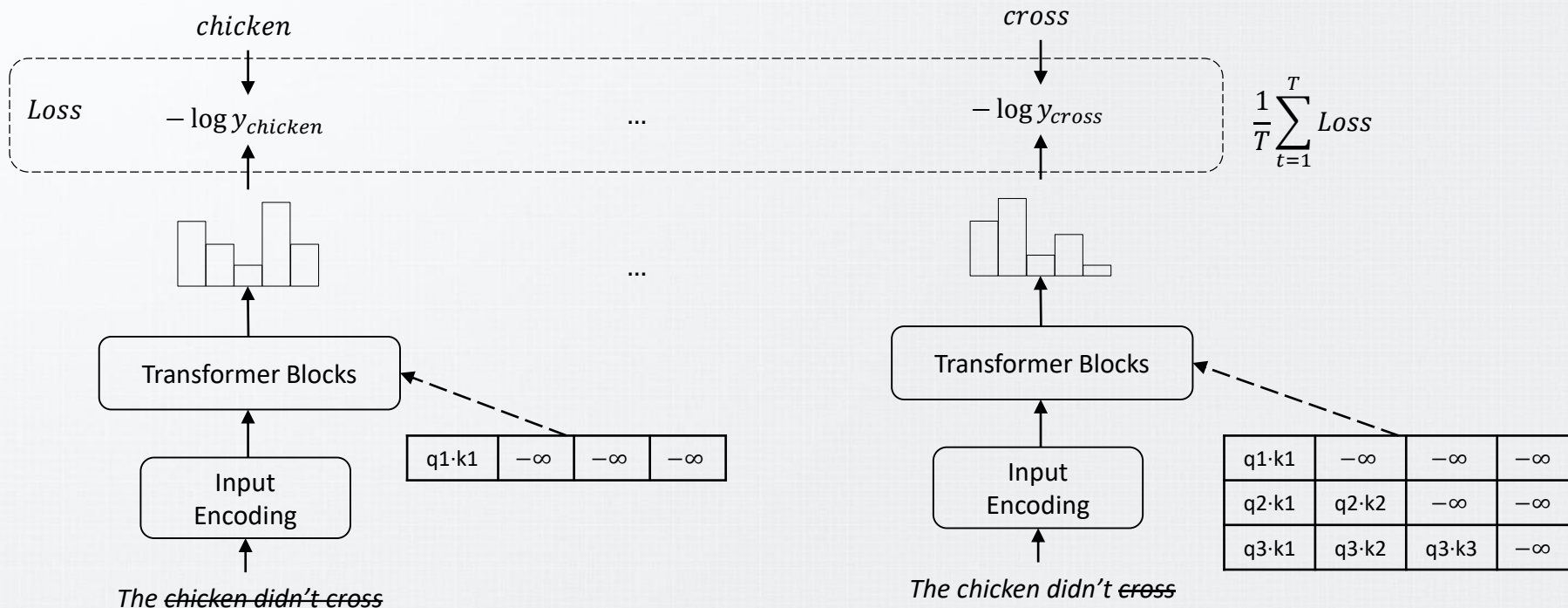
Transformer Architecture



Training Stage

Transformer Architecture

- Next-token prediction objective



Perplexity

Transformer Architecture

- Next-token prediction objective
 - It makes model predictions inherently conditional on **previous context quality**
- **Perplexity** is a widely used **measure** in language modeling
 - It quantifies how well a model predicts a sequence of tokens
 - Given a sequence of tokens $\{y_1, y_2, \dots, y_T\}$, we define the perplexity P in terms of the exponentiation of the average negative log-likelihood of the predicted tokens:

$$P(\{y_1, y_2, \dots, y_T\}) = e^{-\frac{1}{T} \sum_{t=1}^T \underbrace{\log p(y_t | y_{1:t-1}; \theta)}_{\text{Conditional probability of predicting token } y_t \text{ given the previous tokens and parameters } \theta}}$$

Generalizing the conditional term to include a prompt

↓

$$P(y_t | x, y_{1:t-1}; \theta)$$

← Additional context (i.e., a prompt)

[Hands-on](#)



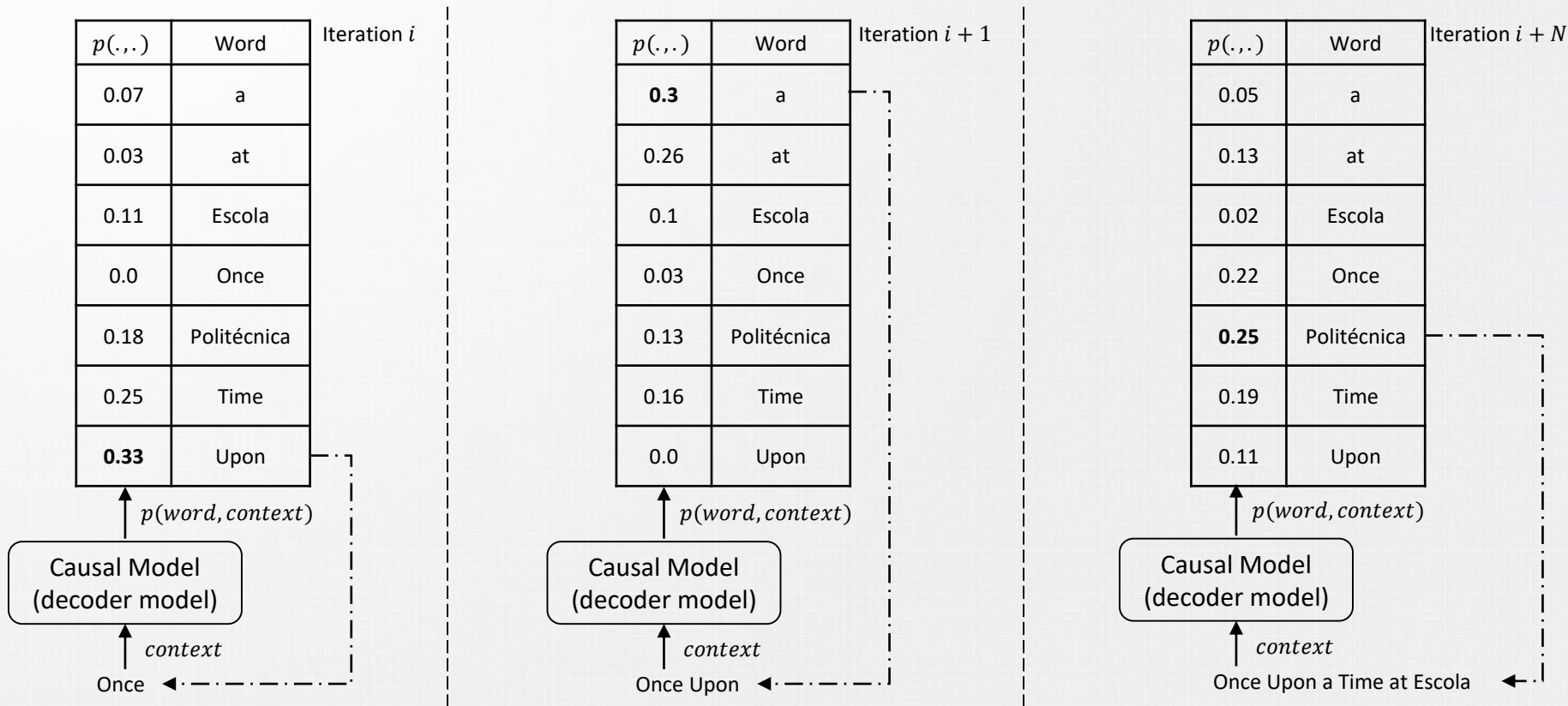
Perplexity

Transformer Architecture

- Intuitively, perplexity reflects how *confused* the model is when predicting the next token
- Low perplexity means confident
 - Accurate predictions
- High perplexity means the model is uncertain
 - The predicted distribution is far from the true sequence

Inference Stage

Transformer Architecture



Architectures Encoder, Decoder and Encoder-Decoder

Transformer Architecture

- Encoder (only)
 - Effective to capture the full representation of a text
 - Non-autoregressive mechanism
 - Example: BERT
- Decoder (only)
 - Generative AI
 - Autoregressive
 - Examples: Llama and GPT
- Encoder-Decoder
 - Generative AI
 - Autoregressive
 - Sequence-to-Sequence

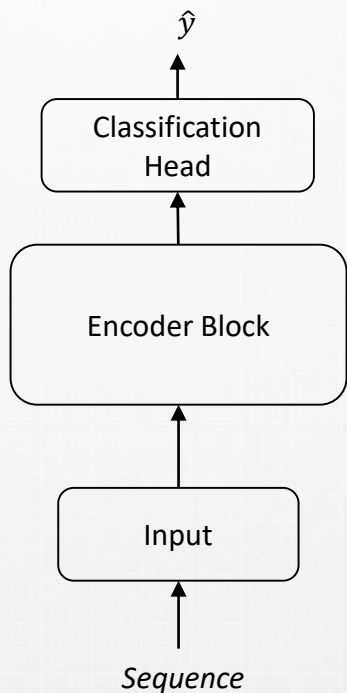
[Hands-on](#)



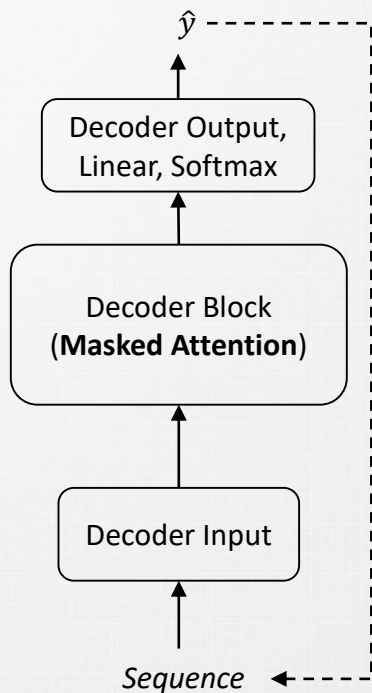
Architectures Encoder, Decoder and Encoder-Decoder

Transformer Architecture

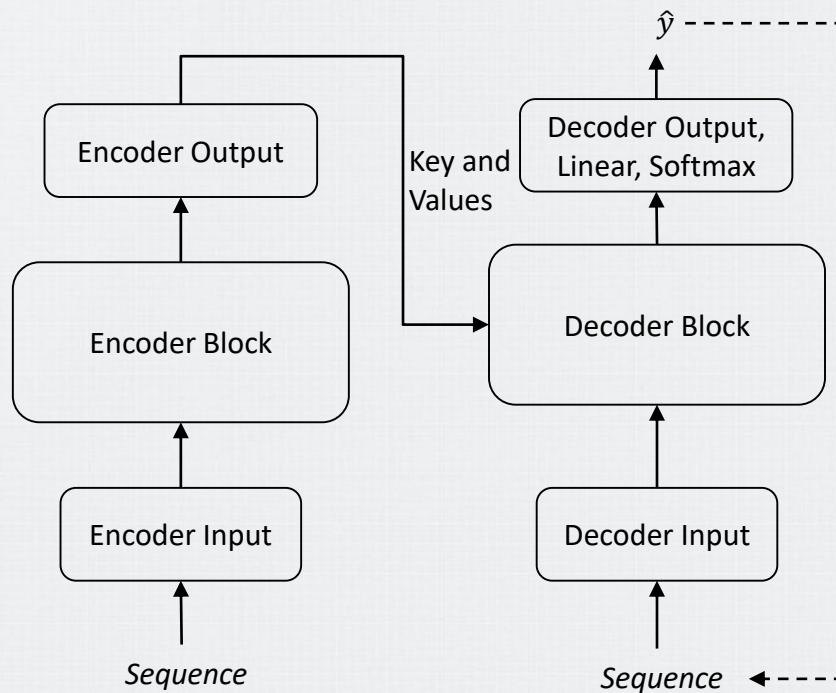
Encoder only



Decoder only



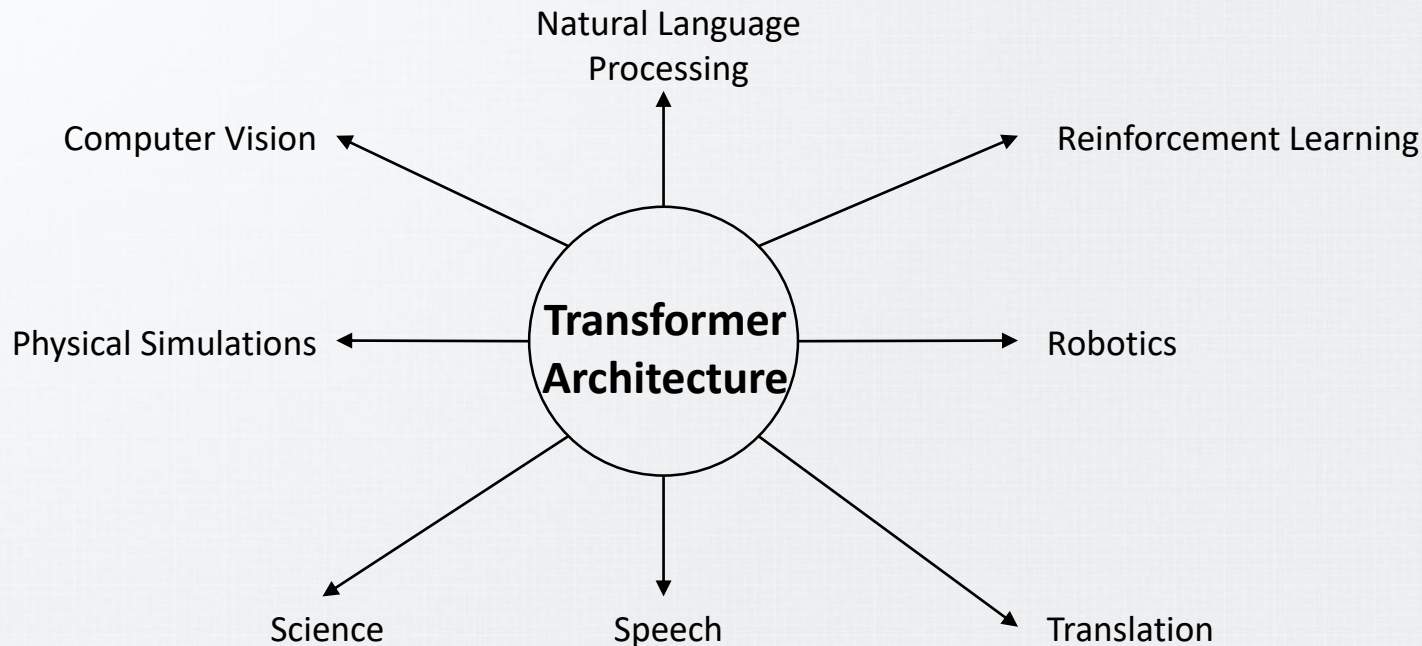
Encoder-Decoder



Multiple Tasks

Transformer Architecture

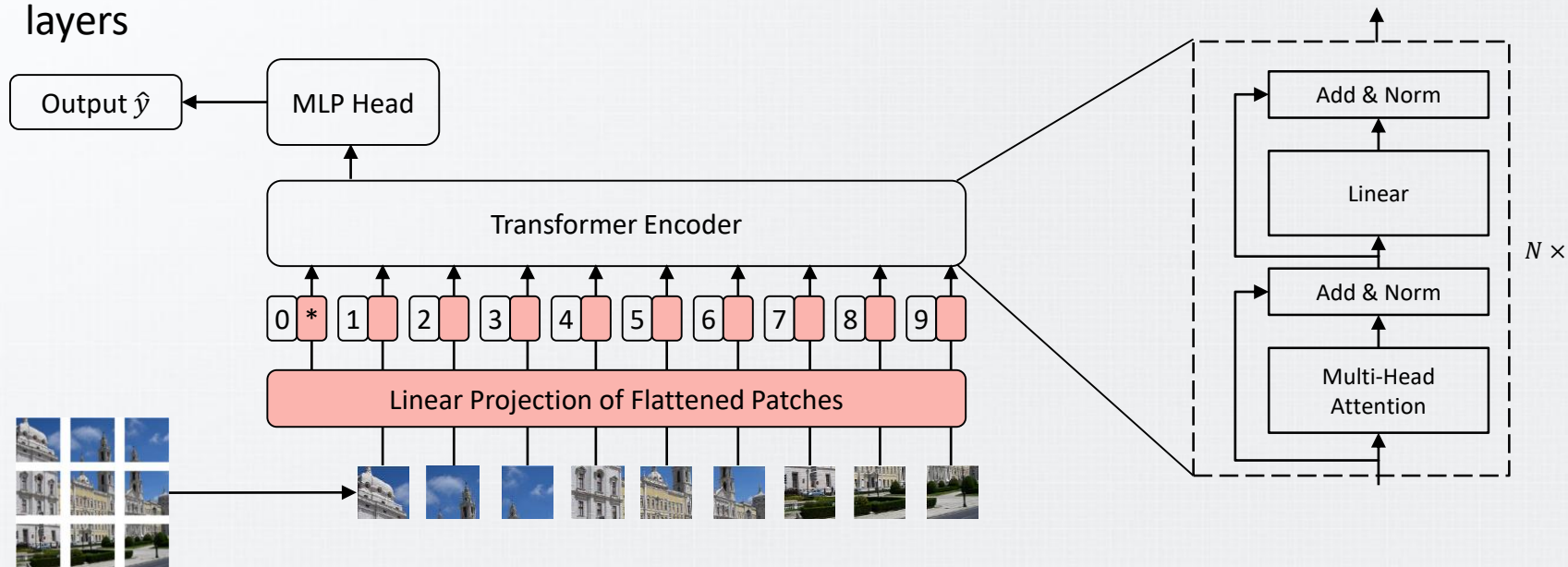
- Copy & Paste (or plug & pRay) the transformer architecture and use it everywhere



Visual Transformer (ViT)

Transformer Architecture

- Transformer architecture for image recognition
- Transformer-based image recognition model that is fully built on the Transformer layers



Data Sensibility

Transformer Architecture

- To achieve state-of-the-art results, ViT requires pre-training on large datasets
 - For example, ImageNet-21k or JFT-300M dataset
- The lack of the typical convolutional inductive bias makes these models more **data-hungry** than common CNNs (Liu et al., 2021)

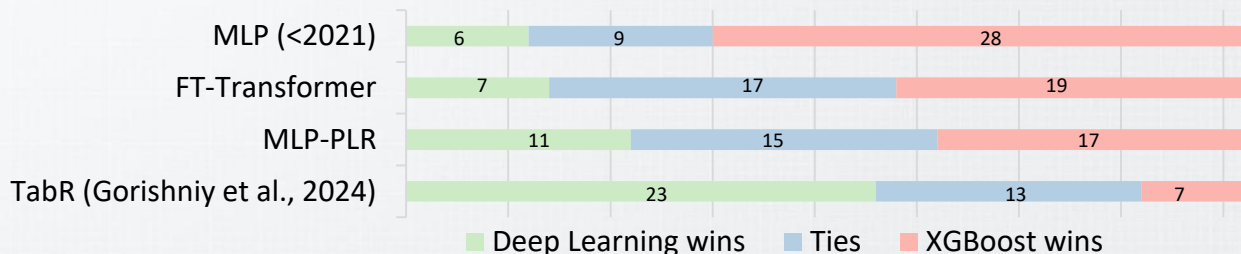
Model	CIFAR-10	CIFAR-100	Flowers102
ViT (variation)	84.19	65.16	31.73
ResNet-50	91.78	72.80	46.92

Source: Liu et al., 2021

Transformer for Tabular Data

Transformer Architecture

- TabPFN (Hollmann et al., 2023)
 - A transformer model for classification that is pre-trained on synthetic data to be applied to unseen datasets, quickly and without hyperparameter tuning
- In the context of tabular data, transformers (and other neural networks) do not achieve state-of-the-art performance (Grinsztajn et al., 2022; Gorishniy et al. 2024)



Hollmann et al. *TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second*. ICLR, 2023
Grinsztajn et al. *Why do tree-based models still outperform deep learning on typical tabular data?* NeurIPS, 2022
Gorishniy et al. *TabR: Tabular Deep Learning Meets Nearest Neighbors*. ICLR, 2024

Foundation Models

Definition

Foundation Models

- Large deep learning models trained on a broad range of data (**web-scale training**) with the capacity to transfer their knowledge to unseen (downstream) tasks
 - Impressive generalization abilities
- Any model is trained on broad data (generally using self-supervision at scale), for which we can adapt it to a wide range of downstream tasks (Bommasani et al., 2021)
- Transformer architectures have ushered in the era of foundation models
 - Most foundation models rely on Transformers

Keys to Foundation Models

Foundation Models

- We could **transfer the large-scale knowledge** captured in these models to target (downstream) tasks by training on a small amount of labeled data
- Keys to Foundation Models
 - Fine-tuning
 - Self-supervised learning

Inductive Bias

Foundation Models

- The tendency of a model to prioritize one solution over another as it extrapolates between data points
- Inductive biases (prior knowledge) show preferences for solutions with certain properties
 - The preference for one choice over others
- Types of inductive bias
 - Locality and translation invariance (CNNs): Bias towards identifying patterns independent of their location within the image
 - Relational (GNNs): Bias towards prioritizing information from neighbors nodes
 - Simplicity Bias (Kirichenko et al., 2023): Bias towards rely on the simple features while ignoring predictive and complex features

Inductive Bias

Foundation Models

- Transformers and MLPs architectures **don't have inductive bias** of convolutional networks
 - By using huge amounts of data, it can surmount this disadvantage
- **Scaling compute** compensates for the lack of inductive bias (Tolstikhin et al, 2021; Bachmann et al., 2023)

Popular Foundation Models

Foundation Models

Model	Task	Number of Parameters	AI Research Organization	Dataset size (Corpus size)
AlphaFold	Protein folding	18M – 60M	DeepMind	3.3B
CLIP	Image classification	63M	OpenAI	400M (image-text pairs)
BERT	Language Understanding Question Answering	110M – 340M	Google	3.3B
BART	Text generation/Understanding	140M – 400M	Facebook	33B
StableDiffusion	Text-to-image generation	890M	LMU Munich + Stability.ai + Eleuther.ai	5B (image-text pairs)
DALL-E	Text-to-image generation	12B	OpenAI	250M (image-text pairs)
GPT-3	Language Understanding	175B	OpenAI	500B
Llama 3	Multitask	8B/70B/405B	Meta	15T Tokens

Llama 3

Foundation Models

- Llama 3 uses a standard, dense Transformer architecture (Vaswani et al., 2017)
 - Regarding architecture, the model does not deviate significantly from Llama and Llama 2
 - Its performance gains are primarily driven by improvements in data quality and diversity
- The model supports context window to 128K tokens

Llama 3 – Training Recipe

Foundation Models

- AdamW optimizer and lower batch size ($4M$ tokens) early in training and increase ($8M$) it subsequently
- The training dynamic is very stable with few loss spikes but without requiring interventions to correct for model training divergence

Component	8B	70B	405B
Number of Layers	32	80	126
Model Dimension	4096	8192	16384
Attention Heads	32	64	128
Activation	SwiGLU (Shazeer et al. 2020)		
Positional Embedding	RoPe (Su et al., 2024)		

Su et al. *Roformer: Enhanced Transformer with Rotary Position Embedding*. Neurocomputing, 2024

Shazeer et al. *GLU Variants Improve Transformer*. ArXiv, 2020

Llama 3 – Computational Issues

Foundation Models

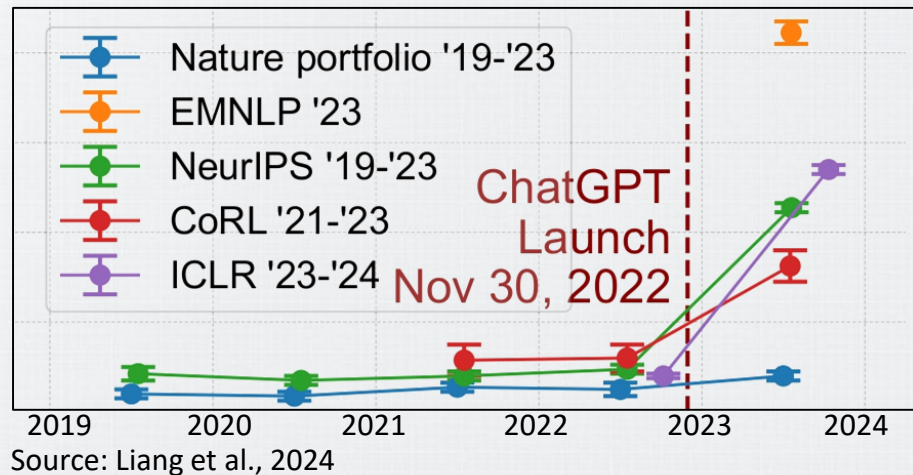
- The largest model training involved up to 16K H100 GPUs distributed globally
- The complexity of 16K GPU training raises potential failure
 - During a **54-day** snapshot period of pre-training, the process experienced a total of 466 job interruptions

Component	Category	% of Interruptions
Faulty GPU	GPU	30.1
Software Bug	GPU	12.9
Network Switch/Cable	Network	8.4
Host Maintenance	Unplanned Maintenance	7.6
...	...	...
Server Chassis	Host	0.5

ChatGPT

Foundation Models

- The impact of ChatGPT on the popularization of Transformers
 - OpenAI released ChatGPT in November 2022, making it the fastest-growing app in history, reaching 1 million users in less than a month and 100 million in less than two months (Amatriain et al., 2023)



Amatriain et al. *Transformer models: an introduction and catalog*. ArXiv 2023

Liang et al. *Monitoring AI-Modified Content at Scale: A Case Study on the Impact of ChatGPT on AI Conference Peer Reviews*. International Conference on Machine Learning (ICML), 2024

ChatGPT

Foundation Models

- The impact of ChatGPT on the popularization of Transformers

CVPR Review Policy

LLM policy: Remember that you can use an **LLM to refine your review text** if you think it is helpful. **But you CAN'T show the paper to an LLM in any way, because doing so is a major violation of policy.** We think we can detect people showing papers to LLMs, and we will prosecute people we catch. Don't do this.

What is the LLM Policy for referees in CVPR 2024?

Details: <https://cvpr.thecvf.com/Conferences/2024/ReviewerGuidelines>

Referees may use any device, including an **LLM, to polish their review wording**, but must vouch for, and be responsible for, the accuracy of the review. It is a significant act of referee misconduct to allow an LLM to see a submission. **PCs interpret showing a submission to an LLM as a deliberate referee violation of confidentiality.**

Clarification on Large Language Model Policy LLM (Details: <https://icml.cc/Conferences/2023/llm-policy>)

Papers that include text generated from a large-scale language model (LLM) such as ChatGPT are prohibited unless the produced text is presented as a part of the paper's experimental analysis.

- The Large Language Model (LLM) policy for ICML 2023 prohibits text produced entirely by LLMs (i.e., “generated”). This does not prohibit authors from using LLMs for **editing or polishing author-written text.**

A Change of Paradigm

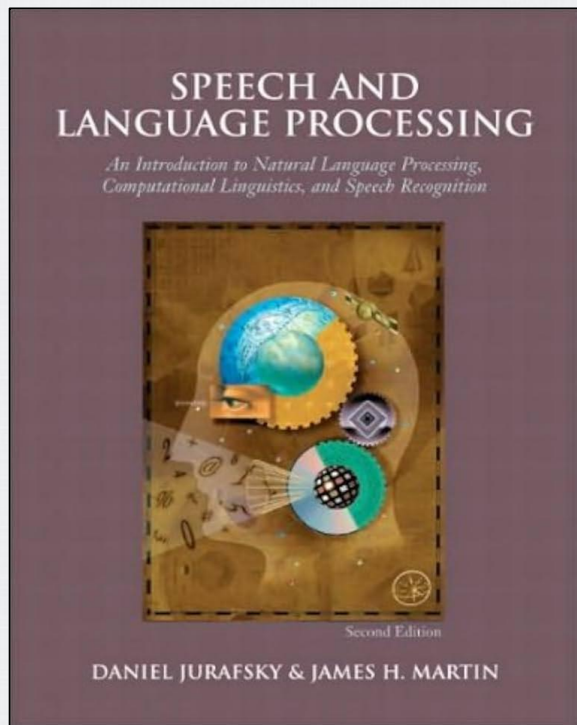
Foundation Models

Classic Machine Learning	Foundation Model Learning
Single model	Model Zoos
Model is trained from scratch	Model is rigid and cannot always be modified (often accessible via API)
Model is the “secret sauce”	Data is the “secret sauce”
Trivial to fine-tune	Non trivial to fine-tune and maintain over time
With sufficient funding, it is possible to establish a local infrastructure	Shortage of accelerators

Bibliography

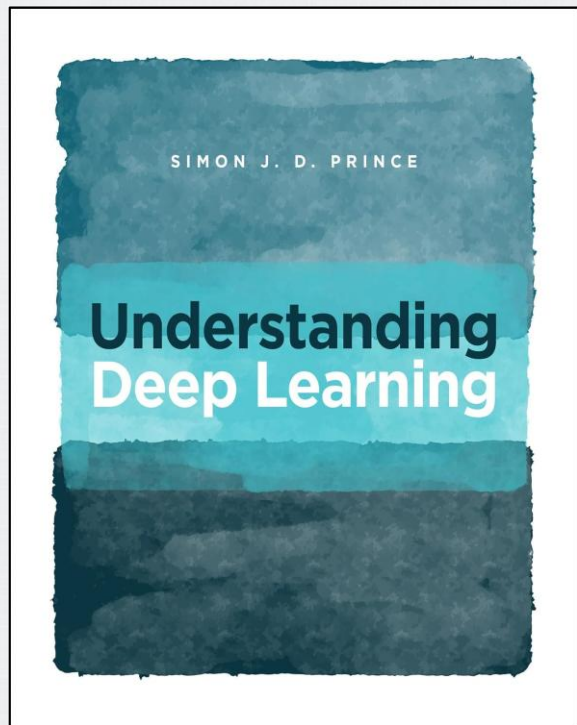
Bibliography

- Understanding Deep Learning
 - Chapter 8
 - Transformers



Bibliography

- Understanding Deep Learning
 - Chapter 12
 - 12.2 Dot-product self-attention
 - 12.3 Extensions to dot-product self-attention
 - 12.4 Transformers



Bibliography

- Vaswani et al. *Attention Is All You Need*. Neural Information Processing Systems (NeurIPS), 2017
- Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. International Conference on Learning Representations (ICLR), 2021
- Liu et al. *Efficient Training of Visual Transformers with Small Datasets*. Neural Information Processing Systems (NeurIPS), 2021



ICLR
International Conference On
Learning Representations

Bibliography

- Bachmann et al. *Scaling MLPs: A Tale of Inductive Bias*. Neural Information Processing Systems (NeurIPS), 2023
- Touvron et al. *ResMLP: Feedforward Networks for Image Classification With Data-Efficient Training*. Pattern Analysis and Machine Intelligence (PAMI), 2023
- Liu et al. *Understanding the Difficulty of Training Transformers*. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020

